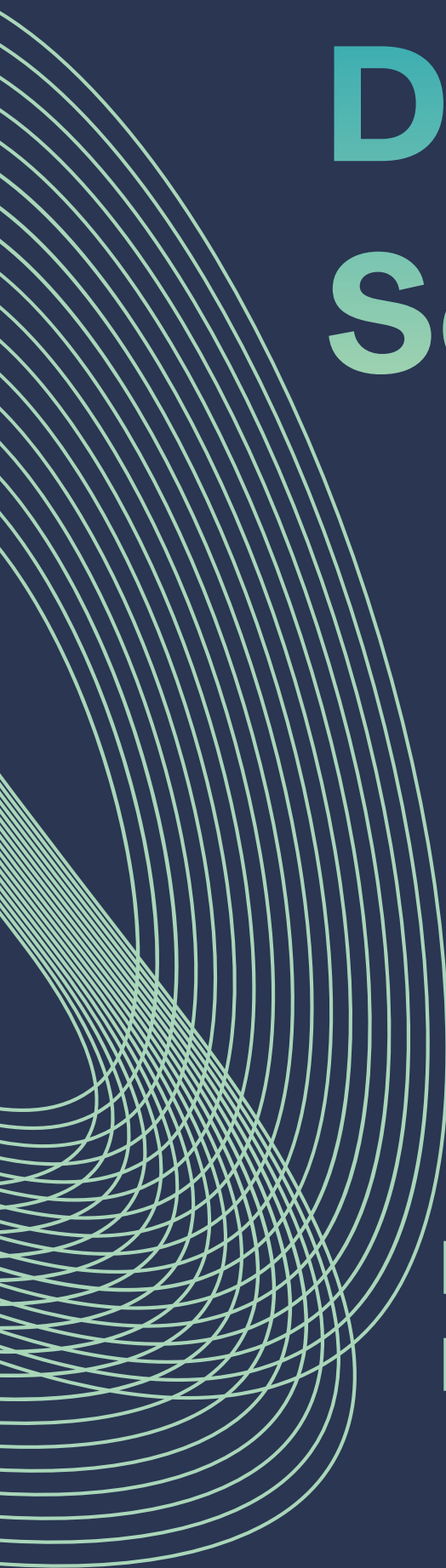




Priority- Driven Task Scheduler

Practical Assignment Report

- Student Name: P. I. Tharindu Gamage
- Student Registration Number:
UWU/ICT/24/093
- Submission Date: July 18th, 2026

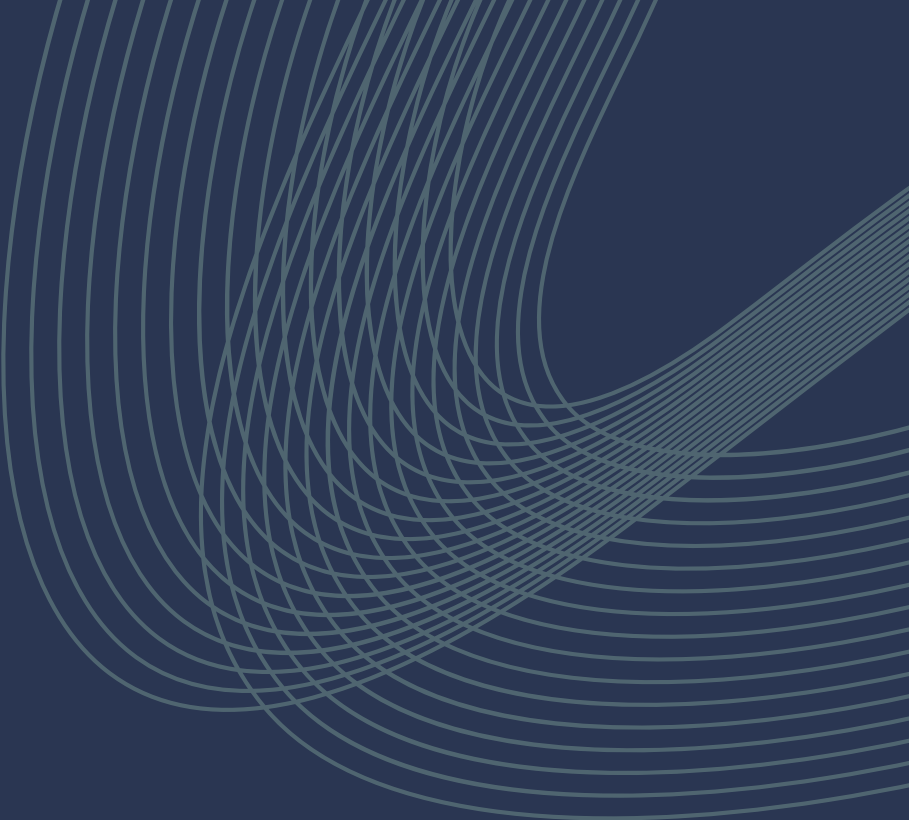


TABLE OF CONTENTS

1.Introduction	3
2.Problem Statement	4
3. System Design	6
4.Data Structures Used	8
5.Algorithms Implemented	10
6.Screenshots	11
7. Conclusion	13

Efficient time management is a critical factor in academic and professional success. To tackle the common issue of task congestion and disorganization, this project presents a command-line Task Scheduler Application developed in Java. The system is designed to help users systematically schedule, prioritize, track, and sort their daily activities.

By applying fundamental computer science theories, the application utilizes three distinct custom-managed data structures. The project satisfies all assignment guidelines while maintaining clean, robust, and validated code execution.



Many traditional to-do list programs store tasks in flat, simple chronological sequences (like standard lists or arrays). This introduces several critical limitations:

- **Inefficient Prioritization:** Users must scan the entire list to find the most important task, resulting in a slow $O(n)$ search time.
- **Lack of Historical Audit Logs:** When tasks are deleted, there is no structural mechanism to review what has been cleared during a session, leaving users without an operational log.
- **Static View Constraints:** Systems often fail to easily alternate between an urgency-based priority model and a strict timeline-based calendar model.
- **No Session Persistence:** Tasks are kept in volatile memory (RAM) and are permanently lost when the console application exits.

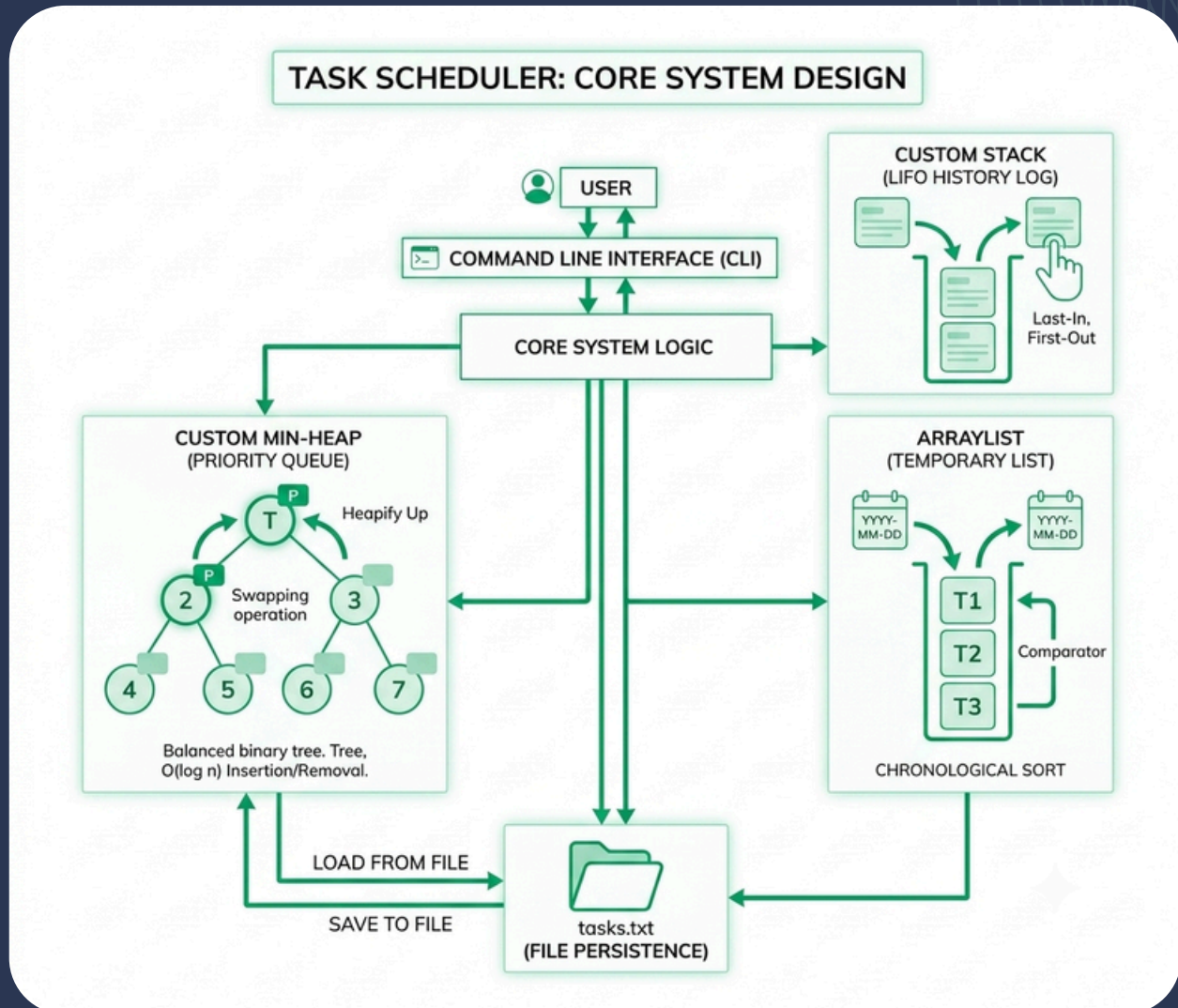
Project Solution

The developed application addresses these gaps by:

- Using a Custom Binary Min-Heap to guarantee instant, $O(1)$ access to the highest-priority task.
- Utilizing a Custom Stack to cache deleted tasks into a Last-In, First-Out (LIFO) history log.
- Implementing Chronological Date Arithmetic and Sorting Algorithms to extract the closest upcoming task and re-order lists by target deadlines.
- Implementing File Persistence using flat-file streams (tasks.txt) to save and reload tasks across runtime sessions.



The system's modular architecture separates user interaction, core data structural processing, and physical file persistence:



Program Workflow

- Initialization: The application starts and calls `loadFromFile()`, parsing elements from `tasks.txt` and building the Priority Queue.

- Insertion: Users input task details (Name, Date, Optional Time, Priority). The system instantiates a Task object, inserts it into the heap, runs heapify-up routines, and updates the text file.
- Analysis & Retrieval: Users can query the closest task for today via day-difference math, or print out full reports arranged either by internal priority hierarchy or by chronological deadline order.
- Deletion & History Tracking: Users choose a specific task index to clear. The system extracts it from the heap, triggers a heapify-down balancing sequence, pushes the deleted item onto the history stack, and updates the text storage file.



The application implements three distinct data structures to satisfy the mandatory assignment guidelines:

A. Custom Min-Heap/Binary Heap (Queues)

- Implementation: Developed using a zero-indexed flat list array structural mapping representation ($\text{leftChild} = 2i + 1$, $\text{rightChild} = 2i + 2$).
- Justification: The primary operation of our scheduler is retrieving the task with the highest priority (e.g., Priority 1). A basic unsorted array takes $O(n)$ to find this element, while a sorted array takes $O(n)$ to insert. A Min-Heap offers the ideal trade-off:
 - Peek (Get Top Priority Task): $O(1)$
 - Insert (Add Task): $O(\log n)$
 - Poll (Remove Task): $O(\log n)$

B. Custom Stack (Stacks)

- Implementation: Array-backed Last-In, First-Out (LIFO) memory structure.
- Justification: The tracking history must access the most recently deleted items first. The Stack perfectly matches this behavior with $O(1)$ push operations, storing deleted records in reverse chronological order for an audit trail view.

C. Sequential Arrays (Arrays)

- Implementation: Used internally to store data sequentially during sorting, mapping, and CLI rendering routines.
- Justification: Provides $O(1)$ index access so the CLI can cleanly map user-selected numbers (e.g., "Delete task #3") directly to corresponding heap objects.

The application utilizes three main algorithmic paradigms to handle task arrangement:

A. Heapify Up (Insertion)

When a new task is added, it is appended to the bottom-right of the tree structure. To restore heap properties, the element is compared to its parent at index $\lfloor (i - 1)/2 \rfloor$. If its priority value is smaller (meaning more urgent, where $1 < 2 < 3$), they swap. This process continues up the tree.

B. Heapify Down (Extraction)

When a task is deleted, the bottom-most element replaces the removed node. To restore heap properties, the node is recursively compared with its left $(2i + 1)$ and right $(2i + 2)$ children, swapping with the smaller of the two until its priority value is smaller than or equal to both children.

C. Chronological Date Sorting (Bubble Sort)

To display tasks in pure calendar sequence without destroying the master priority configuration, a custom sorting routine is written. It loops through a copied collection, reads the string-based dates, parses them into `LocalDate` objects, and executes swaps using the following logic:

If $\text{Date}_j > \text{Date}_{j+1} \implies \text{Swap}(\text{Task}_j, \text{Task}_{j+1})$

This arranges the output linearly from the earliest deadline to the furthest deadline in $O(n^2)$ time complexity.

Figure 1: Main Menu Interface (Showing choices 1 to 7).

```
=====
TASK SCHEDULER SYSTEM
=====
1. Add Task
2. Show Closest Task for Today
3. Show All Tasks (Priority Order)
4. Delete a Task
5. Show All Tasks (Deadline Order)
6. View Deleted Tasks History (Trash Log)
7. Exit
Enter your choice: █
```

Figure 2: Adding a Task (Showing validation handling for input formats like YYYY-MM-DD).

```
Enter your choice: 1

--- Add New Task ---
Enter Task Name: Service car
Enter Due Date (format: YYYY-MM-DD): 2026-07-26
Enter Time (Optional, press Enter to skip): 10:30am
Enter Priority (1 = Most Urgent, 2 = Medium, 3 = Least Urgent): 1

Task added successfully!
```

Figure 3: Closest Task Output (Showing the single task with the absolute minimum number of days separating it from today's date).

```
Enter your choice: 2

--- Closest Task for Today ---
Today's Date: 2026-07-17
The closest task is:
[Priority 2] Buy milk (Due: 2026-07-25, Time: N/A)
(Difference: 8 day(s) away)
```

Figure 4: Priority Order vs. Deadline Order (Side-by-side verification showing heap-sorted priorities vs. date-sorted sequences).

```
Enter your choice: 3
```

```
--- All Scheduled Tasks (Priority Order) ---
```

1. [Priority 1] Service car (Due: 2026-07-26, Time: 10:30am)
2. [Priority 2] Clean the livingroom (Due: 2026-07-28, Time: 8:00am)
3. [Priority 2] Buy milk (Due: 2026-07-25, Time: N/A)
4. [Priority 3] Buy toothbrush (Due: 2026-07-26, Time: N/A)

```
Enter your choice: 5
```

```
--- All Scheduled Tasks (Deadline Chronological Order) ---
```

1. [Priority 2] Buy milk (Due: 2026-07-25, Time: N/A)
2. [Priority 1] Service car (Due: 2026-07-26, Time: 10:30am)
3. [Priority 3] Buy toothbrush (Due: 2026-07-26, Time: N/A)
4. [Priority 2] Clean the livingroom (Due: 2026-07-28, Time: 8:00am)

Figure 5: Deleted History Log (Showing recently deleted entries rendered cleanly in LIFO order from the stack).

```
Enter your choice: 6
```

```
--- Deleted Tasks History (Trash Log - LIFO) ---
```

- [Archived 1] [Priority 2] Buy milk (Due: 2026-07-25, Time: N/A)
- [Archived 2] [Priority 1] Service car (Due: 2026-07-26, Time: 10:30am)

Figure 6: Persistent File (tasks.txt) (Showing how records are saved as plain text).

```
1 Clean the livingroom,2026-07-28,8:00am,2
2 Buy toothbrush,2026-07-26,N/A,3
3
```

This project successfully applies core data structures and algorithms to solve a practical, real-world task management problem. By constructing a custom Min-Heap and LIFO Stack from scratch, the system achieves highly optimized runtimes for its core features:

Operation	Custom Data Structure / Algorithm
Add Task	Min-Heap Insertion
Show Closest Task	Linear Math Absolute Minimum Search
Show All Tasks (Priority)	Min-Heap Poll Mining Loop
Delete Task	Arbitrary Min-Heap Removal Balance
Show All Tasks (Deadline)	Iterative Bubble Sort on Dates
View Deleted History Log	Stack LIFO Iteration